

TokenOps 3 - Staking

Print 

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	DeFi Staking
Timeline	2025-05-20 through 2025-05-27
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	VestingLabs/staking-contracts-v1  #9e0677c 
Auditors	- Julio Aguilar Auditing Engineer - Cameron Biniamow Auditing Engineer - István Böhm Auditing Engineer

Documentation quality	Medium 
Test quality	High 
Total Findings	10  Fixed: 8 Acknowledged: 1 Mitigated: 1

High severity findings ⓘ	0
Medium severity findings ⓘ	1 Fixed: 1
Low severity findings ⓘ	4 Fixed: 3 Mitigated: 1
Undetermined severity findings ⓘ	0
Informational findings ⓘ	5 Fixed: 4 Acknowledged: 1

Summary of Findings

This project implements a modular staking system with two contract types: `Staking` for standard time-based rewards and `StakingWithUnbonding`, which adds an unbonding period before withdrawals. Both support Merkle-based and address-based whitelisting and configurable fees. Factory contracts allow campaign creators to deploy customized staking instances with different parameters.

Overall, the code is well-written, well-documented, and follows best practices. This audit uncovered security issues related to possible DoS from large arrays, access control, incomplete initialization, and input validation. We recommend that the team address these concerns before deployment.

Fix-Review Update: The team addressed all issues by fixing, mitigating or acknowledging them.

ID	TOK-1
DESCRIPTION	DoS While Calculating Rewards
SEVERITY	• Medium ⓘ
STATUS	Fixed

ID	TOK-2
DESCRIPTION	Inconsistent Fee Configuration is Possible

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

TOK-3

DESCRIPTION

Unclaimable Reward Tokens

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

TOK-4

DESCRIPTION

Not Using SafeERC20 for All Token Transfers

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

TOK-5

DESCRIPTION

Inaccurate Max Pool Cap and Lower and Upper Bound Checks in `_stake()`

SEVERITY

• Low ⓘ

STATUS

Mitigated

ID

TOK-6

DESCRIPTION	maxStakingConditions Check Allows One More Condition than Intended
SEVERITY	Informational ⓘ
STATUS	Fixed

ID	TOK-7
DESCRIPTION	Updating Both the Time Unit and Reward Ratio Requires the Creation of Two Staking Conditions
SEVERITY	Informational ⓘ
STATUS	Fixed

ID	TOK-8
DESCRIPTION	Unbonding Period Can Be Changed for Active Unbonding Requests
SEVERITY	Informational ⓘ
STATUS	Acknowledged

ID	TOK-9
DESCRIPTION	Insufficient Parameter Validation
SEVERITY	Informational ⓘ
STATUS	Fixed

--	--

ID	TOK-10
DESCRIPTION	Gas-Inefficient Whitelisting Implementation
SEVERITY	• Informational ⓘ
STATUS	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an

- attempt to identify potential vulnerabilities.
3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
 2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
 3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
 4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Operational Considerations

The contracts require the owners to manually fund the reward pools, introducing an operational overhead and the potential risk of underfunding. If rewards are not topped up in time, staking participants may receive less than expected or no rewards at all, which could impact user trust and protocol engagement. The reward claiming is executed on a first-come, first-served basis. If the rewards run out, users will not be able to claim their unclaimed rewards.

It is possible to deploy `StakingWithUnbonding` contracts exceeding `100%` APY.

The system relies on privileged roles such as the `owner` and `feeCollector`, which centralize control over key functions. The `owner` can update critical parameters like reward ratios, time units, and pool caps, while the `feeCollector` has the authority to withdraw protocol fees and assign the role to another address. This centralization should be carefully managed to mitigate risks from compromised or misused privileges.

Whitelist access is enforced through Merkle proofs and a list of trusted addresses, allowing selective participation. However, this mechanism depends on proper off-chain generation and management of the Merkle tree and the trusted address list. Errors or inconsistencies in the whitelists could lead to eligible users being excluded or unauthorized users being included.

Finally, the codebase does not support `ERC20` tokens with fee-on-transfer or rebase features. Owners should choose compatible tokens to avoid unexpected behavior.

Key Actors And Their Capabilities

`Staking` and `StakingWithUnbonding`

- Owner
 - Can set time unit and reward ratio
 - Can fund the staking contract with tokens for rewards
 - Can withdraw excess rewards or accidentally sent tokens
 - Can set the pool maximum capacity

- Can set the maximum number of staking conditions
 - Can toggle whitelist functionality
 - Can withdraw other tokens (not the staking token)
- Fee Collector
 - Can withdraw accumulated token fees
 - Can withdraw accumulated gas fees
 - Can transfer the fee collector role to another address
- Users (Stakers)
 - Can stake tokens (if within tier bounds)
 - `Staking` contract:
 - Can withdraw staked tokens (if remaining amount meets tier requirements)
 - `StakingWithUnbounding` contract:
 - Can initiate unbonding for staked tokens
 - Can complete unbonding after the required period
 - Can claim rewards based on staking duration and amount
 - Must pay fees (gas or token) for operations if enabled
- Whitelisted Users
 - Can stake tokens if they provide a valid Merkle proof
 - Have the same capabilities as regular users but can bypass whitelist restrictions

`StakingFactory` and `UnboundingStakingFactory`

- Factory Owner
 - Can set default gas and token fees
 - Can set custom fees for specific campaign creators
 - Can set preferred fee types
 - Can enable/disable custom fees
 - Can set fee functions (which operations incur fees)
 - Can set the fee collector address
- Campaign Creator
 - Can deploy new `Staking` or `StakingWithUnbounding` contracts through the factory
 - May have custom fee settings assigned by the factory owner
 - Becomes the owner of the deployed staking contract

Findings

TOK-1 DoS While Calculating Rewards

• **Medium** ⓘ **Fixed**

✓ Update

Fixed in `da37d6c3f5e989aa16c6cccfbfc8d38ebddfa34f` .

The client reduced the initial value of `maxStakingConditions` to 20 and introduced a global cap `MAX_STAKING_CONDITIONS_LIMIT` set to 512. While users still need to loop through all conditions since their last update to calculate rewards, these values

were chosen based on thorough gas usage testing to ensure that even large iterations do not present practical issues.

File(s) affected: `contracts/Staking.sol`

Description: Staker rewards are calculated by iterating over all staker conditions since the staker's last update. If a staker has not staked, withdrawn, or claimed rewards after a sufficiently high number of staker condition updates, the function `_claimRewards()` could run out of gas and fail. Thus, the staker could experience a denial of service and be prevented from interacting with their stake altogether.

While `maxStakingConditions` (default `30`) provides a cap, if the `owner` sets this cap sufficiently high using the `setMaxStakingConditions()` function, transactions involving reward calculation (`stake()`, `withdraw()`, `claim()`) could become too gas-intensive, potentially exceeding block gas limits and preventing the users from executing the affected functions.

Recommendation: Consider adding a function to the `Staking` contract that allows stakers to manually calculate rewards for a specified number of condition updates, giving them control over gas usage. Alternatively, thoroughly evaluate the gas costs of the `_calculateRewards()` function, especially given the default of `30` iterations, and assess whether setting a hard limit on the number of staking conditions would help ensure reliable performance within gas constraints.

TOK-2 Inconsistent Fee Configuration is Possible

• Low ⓘ Fixed

✓ Update

Fixed in `f7a1a7b4510b5cf502ebd1bfd2134730eeadcc56`.

The client fixed this issue by redesigning the `setCustomFee()` function to completely initialize a fee entry in the `_customFees` mapping. Additionally, the functions `setCustomFeeFunctions()`, `setPreferredFeeType()` and `enableCustomFee()` were removed.

File(s) affected: `contracts/lib/FactoryFeeManager.sol`

Description: The `FactoryFeeManager` contract defines the data structure `CustomFee` as follows:

```
struct CustomFee {
    bool enabled;
    ITypes.FeeType preferredFeeType;
```

```

uint256 gasFee;
uint256 tokenFee;
uint8 feeFunctions; // Determines which functions have fees
}

```

However, no function initializes or sets a given entry in the state variable `_customFees` completely:

- The `setCustomFeeFunctions()` function only sets the struct member `feeFunctions` without checking whether the given entry was already enabled or initialized.
- The `setCustomFee()` function sets `enabled` to true if it was disabled before, and only in that case, it will also set the `preferredFeeType`. It will also set either `gasFee` or `tokenFee` depending on the provided `feeType`. If the entry was already enabled before, the function won't update the `preferredFeeType`. Additionally, it does not set the `feeFunctions`.
- The function `setPreferredFeeType()` will set the `preferredFeeType`, but it will also enable the entry if it was disabled before. In this case, all other struct members are uninitialized.
- The `enableCustomFee()` function will only set `enabled` to true if it was false before, potentially leaving all other struct members uninitialized. This means `preferredFeeType` would be `Gas`, and all other values would be zero.

As a consequence, it is possible for an entry in `_customFees` to never be fully initialized, which could result in unexpected behavior and prevent users from interacting with the staking contracts.

Recommendation: We recommend creating a function that fully initializes a custom fee entry and having other functions that only set parts of it if it was already initialized and revert otherwise. Consider adding a `NONE` type to the enum `FeeType` to indicate the uninitialized state.

TOK-3 Unclaimable Reward Tokens

• **Low** ⓘ **Fixed**

✓ Update

Fixed in `201269dfd4bb3be75e27c8c1f22d87ad1bc2684e`.

The client addressed this issue as recommended. There is a new function that requires the amount of rewards to claim: `claimRewards(uint256 amount)`.

File(s) affected: `contracts/Staking.sol`

Description: The `Staking` contract owner funds rewards, and there is no guarantee that there are sufficient rewards to cover a staker's full reward balance. When stakers call

`claimRewards()` , the staker's full reward balance is claimed; however, if the `Staking` contract cannot cover the staker's full reward balance, the staker cannot claim any amount of rewards. Therefore, users with large stakes may be unfairly prevented from claiming their rewards as their reward balance could increase rapidly and exceed the `Staking` contract's allocated rewards balance.

Exploit Scenario:

1. The `Staking` contract is configured to distribute 1% of staked tokens as rewards every seven days.
2. The `Staking` contract reward allocation is 95 `STAKING_TOKEN` .
3. Bob stakes 10,000 `STAKING_TOKEN` .
4. Alice, Joe, and Tom stake 100 `STAKING_TOKEN` each.
5. Bob's rewards are 100 `STAKING_TOKEN` after seven days, while the others have accrued 1 `STAKING_TOKEN` each.
6. Bob cannot claim any portion of his 100 `STAKING_TOKEN` as it exceeds the `Staking` contract's reward allocation of 95.
7. Alice, Joe, and Tom claim their rewards.
8. All stakers continue to accrue rewards, but Bob cannot claim any rewards unless the `Staking` contract reward allocation increases.

Recommendation: Modify the `claimRewards()` function so that stakers can specify an amount of their accrued rewards to claim.

TOK-4 Not Using `SafeERC20` for All Token Transfers

• Low ⓘ Fixed

✓ Update

Fixed in `47c87f8e15308e864a2fb2e4a223917cef227dbc` .

The client fixed this as recommended and replaced all transfer functions with their `safe` counterpart.

File(s) affected: `contracts/StakingWithUnbonding.sol`

Description: The `_claimRewards()` , `completeUnbonding()` and `_stake()` functions transfer `ERC20` tokens using `transfer()` and `transferFrom()` rather than `safeTransfer()` and `safeTransferFrom()` from the OpenZeppelin `SafeERC20` library.

Some `ERC20` tokens may behave differently from the recommended implementation, which can cause issues during token transfers in the smart contract. The `SafeERC20` library ensures that the token transfers return a boolean value or revert upon failure.

Recommendation: Consider using `safeTransfer()` and `safeTransferFrom()` from the `SafeERC20` library to ensure transfers revert on failure.

TOK-5

Inaccurate Max Pool Cap and Lower and Upper Bound Checks in `_stake()`

• Low ⓘ Mitigated

📘 Update

Partially fixed in `e4c5edf18c51cb5edeb0dd9b906b08cb6335d2cd` .

The `_stake()` function properly checks the `poolMaxCap` .

The client provided the following explanation:

If `_stake()` function deducts the fee before checking bounds, which causes issues when users stake via external entities hooked with merkle based flows (eg. merkle distributor `claimAndStake`), they can't send extra tokens if the post-fee amount falls below the lower bound. Since users already meet the lower bound pre-fee, we should validate bounds before fee deduction. This ensures seamless staking via `claimAndStake` and avoids requiring external callers to pre-calculate fee-adjusted amounts.

Description: In the `_stake()` functions, the max pool cap and lower and upper bound checks are performed before the fee is deducted from the `_amount` variable. This can result in small inaccuracies in the checks if the distribution token fee on stake is enabled.

Recommendation: Consider deducting the fee from the staking amount before the checks are performed on it.

TOK-6

`maxStakingConditions` Check Allows One More Condition than Intended

• Informational ⓘ Fixed

✅ Update

Fixed in `07b9d939eacad28f16870ccf890d450380bea986` .

File(s) affected: contracts/Staking.sol

Description: The `_setStakingCondition()` function verifies if `conditionId <= maxStakingConditions` before creating a new staking condition (`conditionId` is 0-indexed):

```
if (conditionId > maxStakingConditions) {  
    revert StakingErrors.ConditionArraySizeLimitReached();  
}
```

If `maxStakingConditions` is intended as the maximum number of conditions (e.g., `N` conditions, IDs `0` to `N-1`), this logic allows `N+1` conditions to be created (IDs `0` to `N`). For instance, if `maxStakingConditions` is `1`, two conditions (ID `0` and ID `1`) can be created. This could lead to slightly higher gas costs than anticipated for reward calculations if the maximum is reached.

Recommendation: Consider verifying that `conditionId < maxStakingConditions`:

```
if (conditionId >= maxStakingConditions) {  
    revert StakingErrors.ConditionArraySizeLimitReached();  
}
```

TOK-7

Updating Both the Time Unit and Reward Ratio Requires the Creation of Two Staking Conditions

• Informational ⓘ

Fixed

✓ Update

Fixed in `9f6b91b40542d145b4a20310e94d453402028430`.

The client added a new function that allows the owner to update multiple parameters in a single transaction.

Description: A `StakingCondition` defines a period during which a specific reward rate applies. The `setTimeUnit()` and `setRewardRatio()` are separate functions for the `owner` to modify the parameters of the staking reward mechanism. When called with new values, each function creates a new `StakingCondition` with the updated parameter. Therefore, updating both the time unit and reward ratio requires the creation of two new staking conditions. However, each added `StakingCondition` increases the gas cost of the reward calculation for the users. It is also noted that the maximum number of staking conditions is limited, increasing the chance of DoS as mentioned in [TOK-1](#).

Recommendation: Consider adding a new function that allows the owner to update all three parameters atomically.

TOK-8

Unbonding Period Can Be Changed for Active Unbonding Requests

• **Informational** ⓘ

Acknowledged

i Update

The client acknowledged the issue and provided the following explanation:

We've chosen to retain the current global-unbonding-period behavior. While it's true that changing the period can unexpectedly delay or accelerate withdrawals for users who have already initiated unbonding, this function is explicitly intended as an emergency "break-glass" mechanism for the admin.

File(s) affected: `contracts/StakingWithUnbonding.sol`

Description: In the `StakingWithUnbonding` contract, the owner can update the global unbonding period at any time via `setUnbondingPeriod()`. This change applies to all users, including those who have already started unbonding but haven't completed it. As a result, users may experience unexpected delays or premature access to their funds if the period is increased or decreased, respectively. This introduces unpredictability and could be misused to delay withdrawals by extending the unbonding period after initiation.

Recommendation: Store the unbonding period in effect at the time each user initiates unbonding, and reference it during `completeUnbonding`. This ensures that updates only apply to new unbonding requests, maintaining consistency and fairness for existing users.

TOK-9 Insufficient Parameter Validation

• **Informational** ⓘ

Fixed

✓ Update

Fixed in `bef5e873d4330e506a80393922fb011b9aa5067b`,
`814270a6a4451f55d7d9a7e5e949f883440b4083` and
`7b8110058ceec4af6f2ebdeaab22adbfefdf6fe3`.

The client added validation for most parameters. For the remaining ones, configuration

is left to the owner's discretion, with warnings to be shown in the UI.

File(s) affected: `contracts/StakingWithUnbonding.sol` , `contracts/Staking.sol` , `contracts/lib/FactoryFeeManager.sol`

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error. The following is a non-exhaustive list of missing input validation:

1. `constructor()` in `Staking` and `StakingWithUnbonding` have insufficient parameter validation:
 1. `MERKLE_ROOT` can be initialized as `bytes32(0)` . Setting a zero value prevents staking. Consider verifying that `params.merkleRoot != bytes32(0)` .
 2. `TIER_LOWER_BOUND` can be initialized to be greater than `TIER_UPPER_BOUND` . Consider verifying that `params.tierLowerBound <= params.tierUpperBound` .
 3. `START_STAKING_DATE` can be initialized to be earlier than the current time. Consider verifying that `block.timestamp <= params.startStakingDate` .
 4. `START_STAKING_DATE` can be initialized to be greater than `END_STAKING_DATE` . Consider verifying that `params.startStakingDate < params.endStakingDate` .
 5. `FEE` can be initialized to a value greater than `BASIS_POINTS ( 10_000 )` . Consider verifying that `params.fee <= BASIS_POINTS` if `params.feeType != ITypes.FeeType.Gas ..`
 6. `FEES_FUNCTIONS` bitmask is not validated and can be set to any `uint8` numbers. Consider verifying that `params.feeFunctions <= 7` .
2. `constructor()` in `StakingWithUnbonding` has insufficient parameter validation:
 1. `maxStakeDuration` can be initialized to a very short duration. Consider verifying that `maxStakeDuration > MIN_DURATION` .
3. `_setStakingCondition` in `Staking` has insufficient parameter validation:
 1. It is possible to configure extremely small or large `timeUnit` . Consider verifying that `_timeUnit >= TIMEUNIT_MIN && _timeUnit <= TIMEUNIT_MAX` .
 2. It is possible to configure a zero or too large `rewardRatioNumerator` . A zero numerator would mean no rewards. Consider verifying that the numeration is in a reasonable range.
4. `setUnbondingPeriod()` in `StakingWithUnbonding` has insufficient parameter validation:
 1. `unbondingPeriod` can be initialized to any value. Consider verifying that `unbondingPeriod <= MAX_UNBONDING_DURATION` .
5. `FactoryFeeManager` :
 1. The `defaultFeeFunctions_` parameter is not validated in `constructor()` . Consider verifying that `defaultFeeFunctions_ <= 7` .
 2. The `newFeeFunctions` parameter is not validated in `setDefaultFeeFunctions()` . Consider verifying that `newFeeFunctions <= 7` .

Recommendation: Consider implementing the above recommendations.

TOK-10 Gas-Inefficient Whitelisting Implementation

✓ **Update**

Fixed in `64ff8e9a7a40c06b8689c28a19ceb3a973cc7a54` and `a35472fb958eb25315159f1f5917386f62e3c85c`.

The Merkle Node now depends solely on the receiver address.

Description: Whitelisting in the `Staking` and `StakingWithUnbonding` contracts is implemented using Merkle proof validation. The leaf `node` is generated in the `stake()` function if whitelisting is enabled using the address of the `_receiver` and the `merkleIndex` and `merkleAmount` parameters:

```
bytes32 node = keccak256(
    abi.encodePacked(merkleIndex, _receiver, merkleAmount)
);

if (!MerkleProof.verifyCalldata(merkleProof, MERKLE_ROOT, node))
    revert StakingErrors.InvalidMerkleProof();
```

However, the `merkleIndex` and `merkleAmount` parameters are unused in the contracts, and including these parameters in the leaf node generation costs more than only using the `_receiver` address.

It is also important to note that the gas cost of Merkle proof validation varies based on tree size. Larger trees require larger Merkle proofs, and therefore, a very large tree can significantly increase the gas costs of the whitelisting validation.

Recommendation: Consider thoroughly testing the gas costs associated with Merkle proof validation to ensure they remain within acceptable limits. If gas usage proves to be a concern, switching to a more efficient whitelisting mechanism, such as using a hashmap or an external contract, could significantly reduce costs.

Auditor Suggestions

S1 Code Improvements

Fixed

✓ **Update**

Fixed in `24eb0439a2ff39c9c4368d77e7d776249daf2f4e` and `d89be6e6bbbc5b857bbc4f97efda327dc67e5d89`.

File(s) affected: `contracts/lib/FactoryFeeManager.sol` , `contracts/Staking.sol` , `contracts/StakingWithUnbonding.sol`

Description: The following is a non-exhaustive list of possible code improvements:

1. `Staking.sol` , `StakingWithUnbonding` :
 - The local variable `withdrawAmount` in the `withdrawExcessRewards()` , `withdrawTokenFee()` , and `withdrawGasFee()` functions are redundant, since `_amount` is not modified within the function and `withdrawAmount` is always equal to it. Consider using only one of the variables.
2. `Staking.sol` :
 - The `nonReentrant` modifier is unnecessary in `fundStakingContract()` at it is a permitted function and follows the Checks-Effects-Interactions patterns.
3. `FactoryFeeManager.sol` :
 - The `setCustomFee()` function takes `newFee` as `uint240`, but the `CustomFee.gasFee` and `CustomFee.tokenFee` struct members are defined as `uint256` . While `uint240` fits into `uint256` , this inconsistency might be unintentional. Consider using `uint256` for the `newFee` parameter to ensure consistency.

Recommendation: Consider applying the above recommendations.

S2 Code Duplication

Fixed

✓ Update

Fixed in `3e930f9d4b35ab5694df5cce37c99d9f99c34aea` .

The client addressed this by introducing a new parent contract, `StakingBase` , which consolidates most of the shared logic between the `Staking` and `StakingWithUnbonding` contracts. This significantly reduces code duplication. A few further improvements could enhance consistency and maintainability:

- The `updatePoolMaxCap()` and `setPoolMaxCap()` functions offer identical functionality; consider removing one to avoid redundancy.
- The external staking functions shared by both `Staking` and `StakingWithUnbonding` can be moved to the `StakingBase` contract to improve code reuse. The internal `_stake()` function should be marked `virtual` in the base and implemented by each derived contract as needed.
- The `claimRewards()` and `claimRewards(_amount)` functions can also be

relocated to the base contract, with `_claimRewards(_amount)` defined as `virtual` and overridden in the child contracts as necessary.

File(s) affected: `contracts/Staking.sol` , `contracts/StakingWithUnbonding.sol`

Description: Code duplication increases maintenance effort, introduces the risk of inconsistencies, and makes the system more error-prone. It reduces readability, slows development, and in the case of Solidity, can result in larger bytecode, leading to higher gas costs and potential deployment limitations. In this codebase, both the `Staking` and `StakingWithUnbonding` contracts share substantial logic. As a result, any bug or update in one contract likely requires a similar change in the other, compounding the effort and risk associated with fixes and upgrades.

Recommendation: Consider extracting shared functionality into an abstract contract that both `Staking` and `StakingWithUnbonding` can inherit from to reduce code duplication.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

The test suite is extensive, with 192 passing tests. To run the tests, use the following commands:

```
forge install
forge test
```

Fix-Review Update: The team has increased the number of tests in the test suite to 269.

```
Warning: optimizer settings have been disabled for accurate coverage
reports, if you encounter "stack too deep" errors, consider using `--
-ir-minimum` which enables viaIR with minimum optimization resolving
most of the errors
```

```
[.] Compiling...
```

```
[.] Compiling 74 files with Solc 0.8.28
```

```
[.] Solc 0.8.28 finished in 4.59s
```

```
Compiler run successful with warnings:
```

```
Warning (2072): Unused local variable.
```

```
--> test/factory/StakingFactory.t.sol:594:3:
|
594 |             IStakingFactory.StakingParams memory params =
IStakingFactory
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
Warning (2072): Unused local variable.
```

```
--> test/factory/StakingFactory.t.sol:635:3:
|
635 |             Staking stakingContract = Staking(stakingAddress);
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
Warning (2072): Unused local variable.
```

```
--> test/staking/MerkleStaking.t.sol:262:3:
|
262 |             uint256 merkleIndex = merkleAddressToIndex[user];
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
Warning (2072): Unused local variable.
```

```
--> test/staking/MerkleStaking.t.sol:316:3:
|
316 |             uint256 merkleIndex = merkleAddressToIndex[user];
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
Warning (2072): Unused local variable.
```

```
--> test/staking/MerkleStaking.t.sol:356:3:
|
356 |             uint256 merkleIndex = merkleAddressToIndex[user];
```


Warning (2072): Unused local variable.

```
--> test/unbonding/edgeCases.t.sol:568:4:
|
568 |             uint64 highAPY,
|             ^^^^^^^^^^^^^^^^^
```

Warning (2072): Unused local variable.

```
--> test/unbonding/edgeCases.t.sol:687:3:
|
687 |             uint256 remainingRewards = limitedRewardStaking
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Warning (2072): Unused local variable.

```
--> test/unbonding/edgeCases.t.sol:989:4:
|
989 |             (uint256 lowStakedAmount, uint256 lowRewards, , , )
= lowAPYStaking
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Warning (2072): Unused local variable.

```
--> test/unbonding/edgeCases.t.sol:991:4:
|
991 |             (uint256 highStakedAmount, uint256 highRewards, , ,
) = highAPYStaking
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Warning (2072): Unused local variable.

```
--> test/unbondingFactory/UnbondingStakingFactory.t.sol:226:3:
|
226 |             UnbondingStakingFactory.CustomFee memory customFee =
factory
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Warning (2072): Unused local variable.

```
--> test/unbondingFactory/UnbondingStakingFactory.t.sol:265:3:
|
265 |             UnbondingStakingFactory.CustomFee memory customFee =
factory
|             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Warning (2072): Unused local variable.

```
--> test/unbondingFactory/UnbondingStakingFactory.t.sol:831:3:
|
831 |             address stakingAddress =
factory.newStakingWithUnbonding(params);
```

```

|          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
Warning (2018): Function state mutability can be restricted to view
--> test/factory/FactoryFeeManager.t.sol:519:2:
|
519 |     function _createDefaultStakingParams()
|     ^ (Relevant source part starts here and spans across
multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> test/factory/StakingFactory.t.sol:112:2:
|
112 |     function test_Constructor() public {
|     ^ (Relevant source part starts here and spans across
multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> test/staking/Staking.t.sol:241:2:
|
241 |     function testContract() public {
|     ^ (Relevant source part starts here and spans across
multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> test/staking/Staking.t.sol:1683:2:
|
1683 |     function testMaxStakingConditionsLimit() public {
|     ^ (Relevant source part starts here and spans across
multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> test/staking/Staking.t.sol:1895:2:
|
1895 |     function test_stakingSettingsShouldReturnCorrectValues()
public {
|     ^ (Relevant source part starts here and spans across
multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> test/unbonding/StakingWithUnbonding.t.sol:1805:2:
|
1805 |     function test_stakingSettingsShouldReturnCorrectValues()
public {
|     ^ (Relevant source part starts here and spans across
multiple lines).

```

```
Warning (2018): Function state mutability can be restricted to view
--> test/unbondingFactory/UnbondingFactoryFeeTests.t.sol:394:2:
|
394 |     function _createDefaultUnbondingParams()
|     ^ (Relevant source part starts here and spans across
multiple lines).
```

```
Warning (2018): Function state mutability can be restricted to view
--> test/unbondingFactory/UnbondingStakingFactory.t.sol:112:2:
|
112 |     function test_Constructor() public {
|     ^ (Relevant source part starts here and spans across
multiple lines).
```

Analysing contracts...

Running tests...

Ran 6 tests for

test/unbonding/EdgeCases2.sol:AdditionalEdgeCaseTests

[PASS] testStakerListFunctions() (gas: 217289)

[PASS] testTransferFeeCollectorRoleToZeroAddress() (gas: 14115)

[PASS] testWithdrawExcessRewardsInsufficientExcessTokens() (gas: 30249)

[PASS] testWithdrawExcessRewardsInsufficientRewards() (gas: 19832)

[PASS] testWithdrawExcessRewardsZeroAddress() (gas: 15413)

[PASS] testWithdrawExcessRewardsZeroAmount() (gas: 17535)

Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 298.63ms (34.16ms CPU time)

Ran 16 tests for

test/factory/FactoryFeeManager.t.sol:FactoryFeeManagerTest

[PASS] test_DefaultFeeTypeIsGas() (gas: 12260)

[PASS] test_DisableCustomFeeWhenNotEnabled() (gas: 43673)

[PASS] test_EnableCustomFee() (gas: 111279)

[PASS] test_FeeTypeDetermination() (gas: 5413419)

[PASS] test_GetCustomFee() (gas: 126840)

[PASS] test_ResetGasFee() (gas: 25123)

[PASS] test_ResetTokenFee() (gas: 25242)

[PASS] test_SetCustomFeeWithInvalidFeeType() (gas: 18349)

[PASS] test_SetDefaultFeeType() (gas: 44847)

[PASS] test_SetInvalidFeeCollector() (gas: 14277)

[PASS] test_SetPreferredFeeTypeWithInvalidFeeType() (gas: 35235)

[PASS] test_UnauthorizedResetGasFee() (gas: 13763)

[PASS] test_UnauthorizedResetTokenFee() (gas: 13894)

[PASS] test_UnauthorizedSetDefaultFeeType() (gas: 14262)

[PASS] test_disableCustomFeeWithInvalidCampaignCreator() (gas:

```
14322)
[PASS] test_setCustomFeeWithInvalidCampaignCreator() (gas: 17850)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in
303.21ms (39.97ms CPU time)
```

```
Ran 12 tests for
test/unbondingFactory/UnbondingFactoryFeeTests.t.sol:UnbondingStakin
gFactoryFeeTests
```

```
[PASS] test_ChangeDefaultFeeType() (gas: 42612)
[PASS] test_CreateWithCustomFeeFunctions() (gas: 4783709)
[PASS] test_CreateWithCustomFees() (gas: 4797240)
[PASS] test_CreateWithDefaultFees() (gas: 4735784)
[PASS] test_DefaultFeeTypeIsGas() (gas: 12327)
[PASS] test_EnableCustomFee() (gas: 131663)
[PASS] test_NonOwnerCannotManageCustomFees() (gas: 22008)
[PASS] test_NonOwnerCannotResetFees() (gas: 17366)
[PASS] test_NonOwnerCannotSetDefaultFeeType() (gas: 14348)
[PASS] test_OwnershipTransfer() (gas: 4732132)
[PASS] test_ResetGasFee() (gas: 17724)
[PASS] test_ResetTokenFee() (gas: 17810)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in
303.89ms (40.42ms CPU time)
```

```
Ran 18 tests for test/staking/StakingBase.t.sol:StakingTest
```

```
[PASS] test_constructorShouldRevertIfInvalidParams() (gas: 354778)
[PASS] test_fundStakingContractWithZeroAmountShouldRevert() (gas:
14312)
[PASS]
test_removeTrustedDistributorShouldDoNothingIfDistributorDoesNotExis
t() (gas: 23533)
[PASS] test_removeTrustedDistributorShouldRemoveDistributor() (gas:
34108)
[PASS] test_removeTrustedDistributorShouldRevertIfAddressIsZero()
(gas: 14249)
[PASS] test_setBatchTrustedDistributorsShouldRevertIfAddressIsZero()
(gas: 15380)
[PASS]
test_setBatchTrustedDistributorsShouldRevertIfArrayLengthIsZero()
(gas: 14526)
[PASS]
test_setWhitelistStatusShouldRevertIfMerkleRootIsNotEmptyAndStatusIs
DisabledOrTrustedDistributors() (gas: 19513)
[PASS] test_setWhitelistStatusShouldRevertSetToMerkleWithEmptyRoot()
(gas: 19184)
[PASS]
test_updatePoolMaxCapShouldRevertIfLessThanCurrentStakedAmount()
```

```
(gas: 223380)
[PASS] test_updatePoolMaxCapShouldUpdateSuccessfully() (gas: 229710)
[PASS] test_withdrawGasFeeShouldRevertIfFeeTypeIsDistributionToken()
(gas: 5221100)
[PASS] test_withdrawGasFeeShouldRevertIfThereAreNoFees() (gas:
5219101)
[PASS] test_withdrawGasFeeWithANonPayableRecipientShouldRevert()
(gas: 5453562)
[PASS] test_withdrawOtherTokenWithZeroAddressShouldRevert() (gas:
14248)
[PASS] test_withdrawOtherTokenWithZeroAmountShouldRevert() (gas:
893606)
[PASS] test_withdrawTokenFeeShouldRevertIfFeeTypeIsGas() (gas:
5221068)
[PASS] test_withdrawTokenFeeShouldRevertIfThereAreNoFees() (gas:
5221201)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in
301.75ms (38.26ms CPU time)
```

Ran 5 tests for

test/unbonding/UtilityFunctions.t.sol:StakingWithUnbondingUtilityTes
t

```
[PASS] testCompleteUnbondingWithNothingUnbonding() (gas: 14169)
[PASS] testGetAddressIndex() (gas: 462600)
[PASS] testGetStakingTokenBalance() (gas: 364824)
[PASS] testIsInStakerList() (gas: 351491)
[PASS] testStakeZeroAmount() (gas: 14703)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in
299.97ms (36.80ms CPU time)
```

Ran 12 tests for

test/staking/FeeConfiguration.t.sol:FeeConfigurationTest

```
[PASS] testCustomFeeFunctionCombinations() (gas: 5460836)
[PASS] testGasFeeOnClaim() (gas: 5503774)
[PASS] testGasFeeOnStake() (gas: 5481236)
[PASS] testGasFeeOnWithdraw() (gas: 5499384)
[PASS] testGasFeeWithdrawal() (gas: 5559227)
[PASS] testNoFees() (gas: 5513464)
[PASS] testTokenFeeCollectorWithdrawal() (gas: 5559027)
[PASS] testTokenFeeOnClaimOperation() (gas: 5511353)
[PASS] testTokenFeeOnStakeOnly() (gas: 5478622)
[PASS] testTokenFeeOnStakeOperation() (gas: 5494129)
[PASS] testTokenFeeOnWithdrawOperation() (gas: 5510272)
[PASS] testTransferFeeCollectorRole() (gas: 5481448)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in
309.96ms (46.38ms CPU time)
```

Ran 26 tests for

test/unbondingFactory/UnbondingStakingFactory.t.sol:UnbondingStakingFactoryTest

[PASS] test_ChangeDefaultFeeCollector() (gas: 4745452)

[PASS] test_ChangeDefaultFeeType() (gas: 4746766)

[PASS] test_Constructor() (gas: 43715)

[PASS] test_ConstructorWithInvalidFeeCollector() (gas: 83843)

[PASS] test_CreateMultipleUnbondingStakingContracts() (gas: 9418018)

[PASS] test_CreateUnbondingStakingContract() (gas: 4768870)

[PASS] test_CreateUnbondingStakingContractWithCustomFees() (gas: 4798766)

[PASS] test_CreateWithExtremelyLargeAPY() (gas: 4731208)

[PASS] test_CreateWithHighAPY() (gas: 4731187)

[PASS] test_CreateWithInvalidDates() (gas: 120772)

[PASS] test_CreateWithInvalidTierBounds() (gas: 118518)

[PASS] test_CreateWithLargeUnbondingPeriod() (gas: 4731430)

[PASS] test_CreateWithMaxAPY() (gas: 4731185)

[PASS] test_CreateWithSameStartAndEndDates() (gas: 120600)

[PASS] test_CreateWithZeroAPY() (gas: 4731151)

[PASS] test_CreateWithZeroMaxStakeDuration() (gas: 174889)

[PASS] test_CreateWithZeroStakingToken() (gas: 120310)

[PASS] test_CreateWithZeroUnbondingPeriod() (gas: 4711506)

[PASS] test_DefaultFeeSettings() (gas: 52018)

[PASS] test_DisableCustomFee() (gas: 110884)

[PASS] test_FactoryOwnerChangeFee() (gas: 4752418)

[PASS] test_MaxStakeDurationAndUnbondingPeriod() (gas: 4733197)

[PASS] test_NonOwnerCannotChangeDefaultFees() (gas: 49887)

[PASS] test_NonOwnerCannotSetFeeCollector() (gas: 23122)

[PASS] test_SetCustomFee() (gas: 104262)

[PASS] test_SetCustomGasFee() (gas: 107272)

Suite result: ok. 26 passed; 0 failed; 0 skipped; finished in 310.85ms (46.75ms CPU time)

Ran 6 tests for

test/factory/FactoryFeeManagerBranch.t.sol:FactoryFeeManagerBranchTest

[PASS] test_AllFeeTypeCombinations() (gas: 15807429)

[PASS] test_DefaultFeeTypeChangeWithCustomFee() (gas: 10537540)

[PASS] test_GetFeeFunctionsInternalBranches() (gas: 15782785)

[PASS] test_GetFeeInternalBranches() (gas: 15861675)

[PASS] test_GetFeeTypeInternalBranches() (gas: 10524517)

[PASS] test_SetCustomFeeWithRawCallInvalidFeeType() (gas: 15503)

Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 10.63ms (9.21ms CPU time)

Ran 3 tests for
test/unbonding/WhitelistFunctions.t.sol:StakingWithUnbondingWhitelis
tTest
[PASS] testStakingToReceiverWithMerkleProof() (gas: 5101200)
[PASS] testStakingWithMerkleProof() (gas: 423897)
[PASS] testWhitelistToggle() (gas: 243041)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 8.25ms
(2.78ms CPU time)

Ran 22 tests for
test/validation/ParameterValidation.t.sol:ParameterValidationTest
[PASS] test_BoundaryValues() (gas: 5224398)
[PASS] test_ConsecutiveParameterUpdates() (gas: 4712128)
[PASS] test_EqualTierBounds() (gas: 5224606)
[PASS] test_FactoryConstructorRevertInvalidFeeFunctions() (gas:
215094)
[PASS] test_FactoryConstructorRevertZeroFeeCollector() (gas: 78698)
[PASS] test_FactoryConstructorWithValidFeeFunctions() (gas:
60463584)
[PASS] test_MaximumUnbondingPeriod() (gas: 4717380)
[PASS] test_RevertInvalidFeePercentageForTokenFee() (gas: 15812)
[PASS] test_RevertInvalidMerkleRootWithWhitelist() (gas: 110658)
[PASS] test_RevertInvalidStakingDates() (gas: 110256)
[PASS] test_RevertInvalidTierBounds() (gas: 110271)
[PASS] test_RevertInvalidUnbondingPeriod() (gas: 162146)
[PASS] test_RevertStartDateInPast() (gas: 110299)
[PASS] test_RevertZeroMaxStakeDuration() (gas: 162118)
[PASS] test_SetDefaultFeeFunctionsValidation() (gas: 27380)
[PASS] test_SetUnbondingPeriodValidation() (gas: 4709485)
[PASS] test_StakingSetRewardRatioValidation() (gas: 5301443)
[PASS] test_StakingSetTimeUnitValidation() (gas: 5304572)
[PASS] test_ValidGasFeeType() (gas: 5288037)
[PASS] test_ValidMerkleRootWithWhitelist() (gas: 5264396)
[PASS] test_ValidStakingCreation() (gas: 5224584)
[PASS] test_ValidUnbondingStakingCreation() (gas: 4717382)
Suite result: ok. 22 passed; 0 failed; 0 skipped; finished in
10.95ms (10.34ms CPU time)

Ran 2 tests for
test/staking/TrustedDistributorStaking.t.sol:SetStakingConditionTest
[PASS] test_stakeFromTrustedDistributor() (gas: 220217)
[PASS] test_stakeFromUntrustedDistributor() (gas: 79360)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.16ms
(388.63µs CPU time)

Ran 20 tests for

```
test/factory/StakingFactory.t.sol:StakingFactoryTest
[PASS] test_ChangeDefaultFeeCollector() (gas: 5256320)
[PASS] test_Constructor() (gas: 43692)
[PASS] test_ConstructorWithInvalidFeeCollector() (gas: 84783)
[PASS] test_CreateContractWithSpecificFeeFlags() (gas: 5254044)
[PASS] test_CreateMultipleStakingContracts() (gas: 10435984)
[PASS] test_CreateStakingContract() (gas: 5268746)
[PASS] test_CreateStakingContractWithCustomFees() (gas: 5309734)
[PASS] test_CreateWithInvalidDates() (gas: 124989)
[PASS] test_CreateWithInvalidTierBounds() (gas: 122733)
[PASS] test_CreateWithLargeRewardRatio() (gas: 21054)
[PASS] test_CreateWithNoFeeFunctions() (gas: 5254021)
[PASS] test_CreateWithZeroPoolMaxCap() (gas: 5220424)
[PASS] test_CreateWithZeroRewardRatioDenominator() (gas: 197200)
[PASS] test_CreateWithZeroStakingToken() (gas: 124548)
[PASS] test_DefaultFeeSettings() (gas: 52061)
[PASS] test_DisableCustomFee() (gas: 107536)
[PASS] test_FactoryOwnerChangeFee() (gas: 5263485)
[PASS] test_NonOwnerCannotChangeDefaultFees() (gas: 49886)
[PASS] test_NonOwnerCannotSetFeeCollector() (gas: 23056)
[PASS] test_SetCustomGasFee() (gas: 103206)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in
13.17ms (7.80ms CPU time)
```

Ran 7 tests for

```
test/staking/SetStakingCondition.t.sol:SetStakingConditionTest
[PASS] testSetStakingConditionBothUnchanged() (gas: 25403)
[PASS] testSetStakingConditionComparedToSeparateCalls() (gas:
10604768)
[PASS] testSetStakingConditionOnlyOwner() (gas: 13436)
[PASS] testSetStakingConditionRewardRatioOnly() (gas: 112700)
[PASS] testSetStakingConditionSuccess() (gas: 116881)
[PASS] testSetStakingConditionTimeUnitOnly() (gas: 111868)
[PASS] test_setStakingConditionsShouldRevertIfStakingEnded() (gas:
26357)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 6.74ms
(5.74ms CPU time)
```

Ran 56 tests for test/staking/Staking.t.sol:StakingTest

```
[PASS] testClaimRewards() (gas: 335066)
[PASS] testClaimRewardsWithInsufficientRewards() (gas: 361276)
[PASS] testConstructorInvalidFeeCollector() (gas: 101657)
[PASS] testConstructorInvalidToken() (gas: 101519)
[PASS] testContract() (gas: 43458)
[PASS] testExploitScenarioUnclaimableRewards() (gas: 774900)
[PASS] testFeeCollectorFunctionsErrors() (gas: 5242323)
```

```
[PASS] testFeeCollectorFunctionsSuccess() (gas: 254066)
[PASS] testFundingContract() (gas: 85761)
[PASS] testMaxConditionsReachedWhenChangingRewardRatio() (gas:
336573)
[PASS] testMaxConditionsReachedWhenChangingTimeUnit() (gas: 335778)
[PASS] testMaxStakingConditionsLimit() (gas: 13583)
[PASS] testOnlyFeeCollectorFunctions() (gas: 23253)
[PASS] testOnlyOwnerFunctions() (gas: 46198)
[PASS] testOwnerFunctionsSuccess() (gas: 229904)
[PASS] testPartialClaimRewards() (gas: 352832)
[PASS] testPartialClaimRewardsMultipleUsers() (gas: 676410)
[PASS] testRewardsAccrual() (gas: 287003)
[PASS] testRewardsWithChangingConditions() (gas: 369622)
[PASS] testSetMaxStakingConditionsAboveLimit() (gas: 17235)
[PASS] testSetMaxStakingConditionsBelowCurrentCount() (gas: 476527)
[PASS] testSetMaxStakingConditionsExactlyAtLimit() (gas: 25297)
[PASS] testSetMaxStakingConditionsSuccess() (gas: 25431)
[PASS] testSetMaxStakingConditionsToZero() (gas: 16075)
[PASS] testSetPoolMaxCapErrors() (gas: 222098)
[PASS] testSetRewardRatioErrors() (gas: 41749)
[PASS] testSetTimeUnitErrors() (gas: 41048)
[PASS] testStakeAboveTierUpperBound() (gas: 85504)
[PASS] testStakeBasic() (gas: 231781)
[PASS] testStakeBelowTierLowerBound() (gas: 85693)
[PASS] testStakeExactTierBounds() (gas: 334200)
[PASS] testStakeExceedingPoolMaxCap() (gas: 305137)
[PASS] testStakeMultipleTimes() (gas: 256589)
[PASS] testStakeWithInvalidParameters() (gas: 5331481)
[PASS] testStakeWithReceiver() (gas: 249550)
[PASS] testStakeWithWhitelistEnabled() (gas: 104204)
[PASS] testStakeZeroAmount() (gas: 80808)
[PASS] testWithdrawBasic() (gas: 277455)
[PASS] testWithdrawExactAmount() (gas: 245958)
[PASS] testWithdrawExcessRewards() (gas: 128450)
[PASS] testWithdrawExcessRewardsErrors() (gas: 111020)
[PASS] testWithdrawExcessRewardsFromExcessTokens() (gas: 140209)
[PASS] testWithdrawLeavingBelowTierLowerBound() (gas: 5421906)
[PASS] testWithdrawMoreThanStaked() (gas: 220320)
[PASS] testWithdrawRemainingAtLowerBound() (gas: 270447)
[PASS] testWithdrawWithNoRewardsAccrued() (gas: 276550)
[PASS] testWithdrawZeroAmount() (gas: 217022)
[PASS]
test_availableRewardsShouldNotCalculateRewardsWhenNothingStaked()
(gas: 267183)
[PASS] test_claimRewardsShouldRevertWhenInsufficientRewards() (gas:
270990)
```

```
[PASS] test_claimRewardsShouldRevertWhenNoRewardsAvailable() (gas: 225367)
[PASS] test_getAddressIndex() (gas: 224209)
[PASS] test_rewardsShouldNotAccrueWhenStakingPeriodEnded() (gas: 238112)
[PASS] test_stakeShouldRevertIfExceedsUpperBoundAfterFee() (gas: 185303)
[PASS] test_stakeShouldSuccessIfBelowMaxCap() (gas: 227753)
[PASS]
test_stakeWithTwoParametersShouldRevertIfRecipientIsZeroAddress()
(gas: 105370)
[PASS] test_stakingSettingsShouldReturnCorrectValues() (gas: 88971)
Suite result: ok. 56 passed; 0 failed; 0 skipped; finished in
10.94ms (32.58ms CPU time)
```

Ran 33 tests for

test/unbonding/StakingWithUnbonding.t.sol:StakingWithUnbondingTest

```
[PASS] testAPYRewardsAccuracy() (gas: 9950539)
[PASS] testClaimRewardsNoRewards() (gas: 223097)
[PASS] testConstructorValidation() (gas: 166724)
[PASS] testGasFeeCollection() (gas: 5076686)
[PASS] testMultiUserInteractions() (gas: 618723)
[PASS] testMultipleStakesWithRewardCap() (gas: 272057)
[PASS] testPartialClaimRewards() (gas: 300754)
[PASS] testPartialClaimRewardsExceedsAvailable() (gas: 254379)
[PASS] testPartialClaimRewardsMultipleUsers() (gas: 639908)
[PASS] testPartialClaimZeroAmount() (gas: 271545)
[PASS] testPoolMaxCapLimit() (gas: 356257)
[PASS] testSetPoolMaxCap() (gas: 229433)
[PASS] testSetUnbondingPeriod() (gas: 37727)
[PASS] testStakeWithReceiver() (gas: 238572)
[PASS] testStakingAndRewards() (gas: 271502)
[PASS] testStakingBounds() (gas: 327683)
[PASS] testStakingPeriodConstraints() (gas: 5052892)
[PASS] testToggleUnbondingPeriod() (gas: 278332)
[PASS] testTokenFeeCollection() (gas: 5023417)
[PASS] testTransferFeeCollectorRole() (gas: 43594)
[PASS] testUnbondingEdgeCases() (gas: 267185)
[PASS] testUnbondingPeriod() (gas: 298740)
[PASS] testWhitelistFunctionality() (gas: 393023)
[PASS] testWithdrawExcessRewards() (gas: 306931)
[PASS] testWithdrawOtherToken() (gas: 929542)
[PASS] testZeroUnbondingPeriod() (gas: 257773)
[PASS] test_stakeShouldRevertIfExceedsUpperBoundAfterFee() (gas: 185627)
[PASS] test_stakeShouldSuccessIfBelowMaxCap() (gas: 227923)
```

```
[PASS]
test_stakeWithSpecifiedReceiveShouldRevertIfWhitelistIsEnabled()
(gas: 80816)
[PASS] test_stakeWithSpecifiedReceiver() (gas: 217863)
[PASS]
test_stakeWithSpecifiedReceiverShouldRevertIfNotTrustedDistributor()
(gas: 63465)
[PASS]
test_stakeWithSpecifiedReceiverShouldRevertIfReceiverIsZeroAddress()
(gas: 24854)
[PASS] test_stakingSettingsShouldReturnCorrectValues() (gas: 66593)
Suite result: ok. 33 passed; 0 failed; 0 skipped; finished in
30.75ms (27.74ms CPU time)
```

Ran 16 tests for

test/unbonding/edgeCases.t.sol:StakingWithUnbondingEdgeCasesTest

```
[PASS] testCreateLimitedRewardsStaking() (gas: 5107957)
[PASS] testDifferentAPYValues() (gas: 9928084)
[PASS] testGasFeeWithdrawal() (gas: 5039550)
[PASS] testMultipleUnbondingOperations() (gas: 337949)
[PASS] testMultipleUsersCompleteUnbonding() (gas: 630187)
[PASS] testMultipleUsersInitiateUnbonding() (gas: 716306)
[PASS] testPartialFeeWithdrawalAndTransfer() (gas: 4999071)
[PASS] testPoolParameterChanges() (gas: 5292575)
[PASS] testProportionalRewardsAfterUnbonding() (gas: 641268)
[PASS] testReStakingAfterUnbonding() (gas: 325016)
[PASS] testRewardClaimingAfterReplenish() (gas: 5160909)
[PASS] testRewardsDepletionByFirstUser() (gas: 5125765)
[PASS] testSmallAmountStakingAndUnbonding() (gas: 4945084)
[PASS] testStakingPeriodBoundaries() (gas: 5084707)
[PASS] testTokenFeeWithdrawal() (gas: 5022942)
[PASS] testUnauthorizedAccess() (gas: 50264)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in
26.42ms (24.92ms CPU time)
```

Ran 8 tests for test/staking/MerkleStaking.t.sol:MerkleStakingTest

```
[PASS] testDisableWhitelistAndStake() (gas: 237651)
[PASS] testFuzzStakeWithAnyMerkleAmount(uint256) (runs: 256, μ:
246283, ~: 246350)
[PASS] testMultipleUsersStakeWithMerkleProof() (gas: 486189)
[PASS] testRegularStakeWithWhitelistEnabled() (gas: 92110)
[PASS] testStakeWithInvalidMerkleProof() (gas: 110551)
[PASS] testStakeWithMerkleProofToDifferentReceiver() (gas: 110577)
[PASS] testStakeWithValidMerkleProof() (gas: 259157)
[PASS]
test_StakeWithSpecifiedReceiverShouldFailIfWhitelistIsEnabled()
```

```
(gas: 83043)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in
105.80ms (110.66ms CPU time)

Ran 1 test for
test/staking/CalculateRewardsGasBenchmark.t.sol:CalculateRewardsGasB
enchmarkTest
[PASS] testCompareAllFunctionGasCosts() (gas: 359192145)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in
467.12ms (204.57ms CPU time)

Ran 18 test suites in 518.39ms (2.82s CPU time): 269 tests passed, 0
failed, 0 skipped (269 total tests)
```

Code Coverage

The code coverage is strong, exceeding 90% across most metrics, with branch coverage slightly lower at around 87%. We recommend adding more tests covering both happy and edge cases to further improve coverage and ensure robustness. To get the coverage results, use the following commands:

```
forge install
forge coverage
```

Fix-Review Update: The team has increased the code coverage test to 100%.

File	% Lines	% Statements	% Branches
contracts/Staking.sol	100.00% (223/223)	100.00% (229/229)	100.00% (48/48)
contracts/StakingWithUnbonding.sol	100.00% (200/200)	100.00% (209/209)	100.00% (43/43)
contracts/factory/StakingFactory.sol	100.00% (8/8)	100.00% (11/11)	100.00%
contracts/factory/UnbondingStakingFactory.sol	100.00% (8/8)	100.00% (11/11)	100.00%
contracts/lib/FactoryFeeManager.sol	100.00% (66/66)	100.00% (55/55)	100.00% (10/10)

File	% Lines	% Statements	% Branches
contracts/lib/StakingBase.sol	100.00% (134/134)	100.00% (140/140)	100.00% (38/38)
Total	**100.00% (639/639)**	**100.00% (655/655)**	**100.00% (139/139)

Changelog

- 2025-05-27 - Initial report
- 2025-05-30 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way,

including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

